

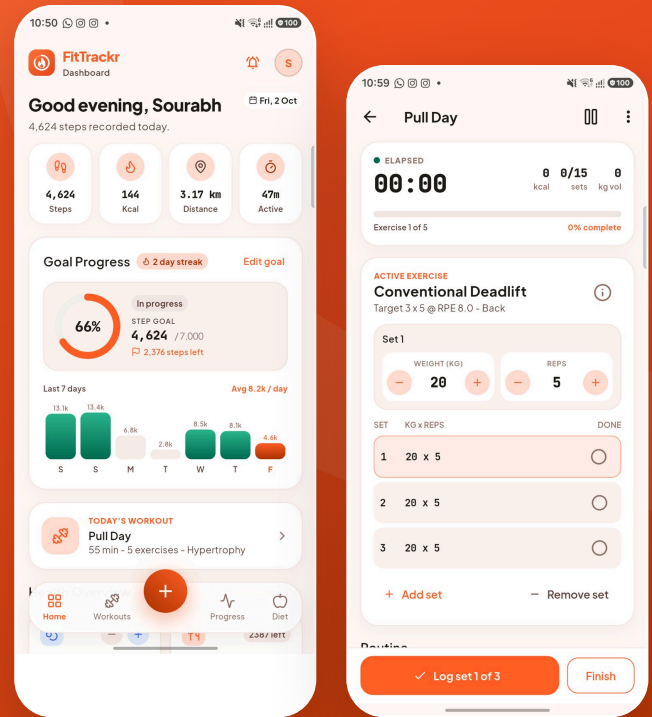


FitTrackr

Offline-first fitness & workout tracker

Project Report: A Flutter Fitness Tracking App

Workouts, step counting, progress analytics and diet suggestions in one Material 3 app, built in Flutter and Dart.



SUBMISSION DETAILS

Student	Sourabh Yadav
Roll number	150096724013
Cohort	MZ
Programme	B.Tech CSE, ITM Skills University
Subject	Cross Platform Application (Flutter App Development)
Application	Application Name 9: FitTrackr
Problem statement	Problem Statement 59
Version	v1.1.0 (build 2), Android
Source code	github.com/YadavSourabhGH/FitTracker
Figma design	figma.com/design/CvAN8r31C30rYkGJ2VwJEt/FitTrackr
APK download	github.com/YadavSourabhGH/FitTracker/releases/latest

Contents

1	Problem statement and objectives	3
2	Solution overview and technology stack	4
3	System architecture	5
4	User journey and Figma design	6
5	Application screens	7
6	Data models and database	13
7	State management and core logic	14
8	Integration, styling and theming	15
9	Testing, CI/CD and deliverables	16
10	Limitations, future scope and conclusion	17

Abstract

People who train usually juggle several apps: one for workouts, one for steps, one for food. FitTrackr brings these into a single Flutter app. A user can pick a workout plan, log every set with a live session clock and rest timer, see steps counted by the phone's hardware sensor, review progress charts, and track meals against a personal calorie target with goal-based diet suggestions.

The app is offline-first. All records are stored on the device in SQLite, the profile and preferences in SharedPreferences, and state is managed with Riverpod. Food search uses the Open Food Facts REST API, with an offline food list as a fallback. Optional Android Health Connect sync adds steps, heart rate, sleep and SpO2 from other apps and watches. The interface follows Material 3 and the approved Figma design, and the project ships with unit, database and widget tests plus a GitHub Actions pipeline that builds a signed release APK.

Project at a glance

4 main tabs: Home, Workouts, Progress, Diet	18 screens and sheets in the user journey	10 SQLite tables, schema version 2	9 built-in workout plans
29 exercises with technique cues	34 offline foods + Open Food Facts search	19 unit, database and widget tests	~12k lines of Dart across 76 files

1 Problem statement and objectives

Problem Statement 59: fitness users need to organise workouts, monitor daily steps, understand progress, and access diet suggestions. FitTrackr consolidates these activities into a Flutter-based fitness tracking interface.

Objectives and how FitTrackr meets them

Objective	Requirement	Implementation in FitTrackr
UI / Widgets	Dashboard, workout plans, workout detail, step counter, progress charts, diet suggestions. Cards, Lists, ProgressIndicators, Charts, Tabs, Filters, navigation.	All six screens built. Cards and lists throughout; linear and circular progress indicators; <code>fl_chart</code> bar and line charts; Track Meals / Suggestions tabs; <code>ChoiceChip</code> filters; custom floating navigation bar.
Styling / Theming	Material 3, fitness-oriented hierarchy, clear activity states, typography, spacing, icons, responsive layout.	<code>useMaterial3: true</code> , custom colour scheme and 16 component themes, bundled Plus Jakarta Sans and JetBrains Mono fonts, Lucide icons, status chips (In progress, Missed, Today, Live), clamped text scaling.
Dart / Flutter logic	Models for workouts, steps, progress, diet; completion, step totals, progress calculations, chart data, state, navigation.	Immutable models with <code>fromMap</code> / <code>copyWith</code> ; set completion with auto-advance; daily and hourly step aggregation; BMR, TDEE, MET, Epley 1RM, streak and XP calculations; weekly chart grouping; Riverpod state; Navigator routes.
Figma	Journey: dashboard → workout plan → workout tracking → steps → progress charts → diet suggestions.	Designed in Figma and implemented screen for screen (section 4).

Expected outcomes

- A functional Flutter prototype demonstrating the core fitness and workout tracking workflow.
- Structured, interactive, maintainable and responsive screens using Dart and Flutter concepts.
- The approved Figma flow translated into a consistent Material 3 interface with widgets, logic, validation and state handling.
- A complete start-to-finish journey: select a workout, record progress, track steps, review charts, view diet suggestions.

Scope

FitTrackr targets Android phones (tested on a Samsung Galaxy M35 5G) and runs fully offline. Internet is only used for optional online food search. One user profile is stored per device; no account is required.

2 Solution overview and technology stack

FitTrackr is organised around four tabs. Each tab answers one of the user needs in the problem statement.

1 Home dashboard

Today's steps, calories, distance and active minutes; step-goal ring and streak; a 7-day step chart; water and calories eaten; today's planned workout; Health Connect vitals; a coach tip chosen from the day's data.

2 Workouts

Nine plans with 29 exercises, filters by category and difficulty, an editable weekly plan, workout detail with technique cues, a live session logger (weight, reps, rest timer, pause), a summary with personal records, and history.

3 Progress

Step counter ring, hourly intensity, 7 and 30-day step history with the goal line, streak tracking and day browsing. Training analytics: weekly volume, estimated 1RM trends, and a body-weight chart.

4 Diet

Personal calorie and macro targets from BMR and activity level; meals logged by Breakfast, Lunch, Dinner and Snack; food search (offline list, recent foods, Open Food Facts); goal-based diet suggestions with filters.

Also included: onboarding with a validated profile form, profile with level, XP and 10 achievements, local reminders, Health Connect settings, and a full data reset.

Technology stack

Area	Technology	Used for
Framework / language	Flutter, Dart 3 (null safety)	Single codebase UI and logic
State management	<code>flutter_riverpod 3</code>	Providers, Notifiers, AsyncNotifiers, dependency injection
Local database	<code>sqflite, path</code>	Workouts, sessions, sets, steps, meals, water, weight
Key-value storage	<code>shared_preferences</code>	Profile, goals, reminders, weekly schedule, step baselines
Sensors / health	<code>pedometer, permission_handler, health</code>	Hardware step counter, activity permission, Health Connect
Networking	<code>http</code>	Open Food Facts REST API (JSON)
Charts	<code>fl_chart, CustomPainter</code>	Bar and line charts, step rings and gauges
Notifications	<code>flutter_local_notifications, timezone</code>	Daily workout, hydration and step reminders
Other	<code>intl, share_plus, lucide_icons_flutter</code>	Number formatting, sharing a workout recap, icons
Testing / CI	<code>flutter_test, sqflite_common_ffi</code> , GitHub Actions	Unit, database and widget tests; automated release APK

3 System architecture

The code follows a layered, feature-organised structure. Screens never touch storage directly: they read Riverpod providers, providers call repositories, and repositories talk to the database, sensors and network.

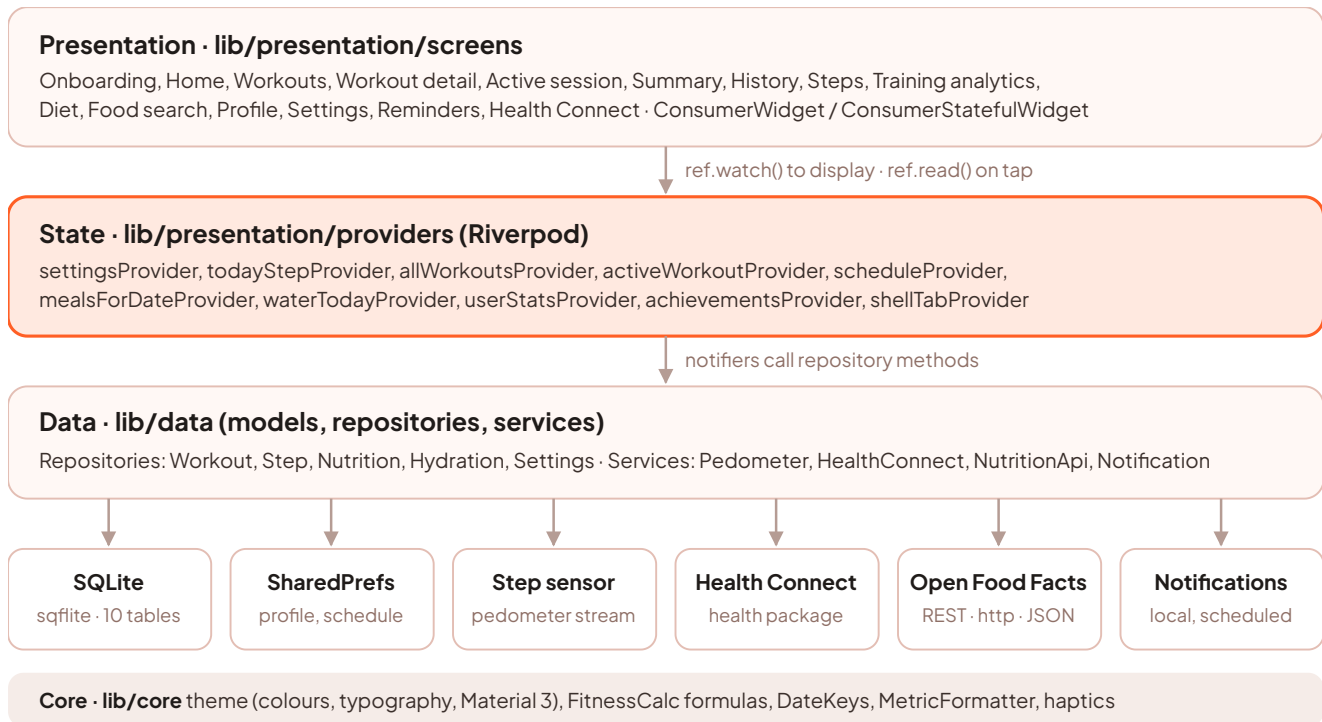


Figure 1. Layered architecture. Data flows down as method calls and back up as model objects.

Folder structure

```

lib/
├─ main.dart           // startup, ProviderScope
├─ core/
│ └─ theme/           // AppTheme, colours,
│   fonts
│   └─ utils/         // FitnessCalc, DateKeys
│     └─ constants/
├─ data/
│ └─ local/           // DatabaseHelper, seed
data
│ └─ models/         // Workout, UserSettings
...
│ └─ repositories/   // SQL + prefs access
│   └─ services/     // sensor, API,
notifications
├─ presentation/
│ └─ providers/       // Riverpod state
│   └─ common_widgets/ // ui_kit, charts, nav
bar
└─ screens/         // one folder per feature
  
```

Design decisions

- **Repository pattern.** Each repository hides SQL behind methods such as `startSession()` or `history(days)`.
- **Dependency injection.** Repositories accept an optional `DatabaseHelper`; Riverpod providers supply them to the UI. Tests swap in an in-memory database.
- **Singletons.** One database connection (`DatabaseHelper.instance`) and one notification service.
- **Offline-first.** Every feature works without internet; network is only used for optional food search.
- **Startup.** `main()` loads settings, schedules reminders, removes abandoned sessions older than 12 hours, then runs the app with `ProviderScope`.

4 User journey and Figma design

The Figma design defines the journey from the fitness dashboard to a workout plan, workout tracking, steps, progress charts and diet suggestions. The app implements that flow with the same screens and order.



Figure 2. Start-to-finish user journey implemented in the app.

Navigation map

Entry point	Destination and mechanism
First launch	Onboarding form → permissions screen (pushReplacement) → main shell (pushAndRemoveUntil , so Back can't return to setup)
Floating nav bar	Home, Workouts, Progress, Diet in an IndexedStack (tabs keep their state). The selected tab lives in shellTabProvider , so any screen can switch tabs.
Centre "+" button	Quick actions sheet: start today's workout, log a meal, add a glass of water, refresh activity
Workout card	Navigator.push to Workout detail, passing the Workout object in the constructor; Start → Active session → Summary
Progress tab card	Training analytics (volume, 1RM and body-weight charts)
Header avatar / bell	Profile (then Settings, Edit profile) / Reminders
Back button	PopScope returns to Home from other tabs before the app closes; an active workout keeps running and can be resumed from a banner

Figma design: [figma.com/design/CvAN8r31C30rYkGJ2VwJEt/FitTrackr](https://www.figma.com/design/CvAN8r31C30rYkGJ2VwJEt/FitTrackr)

The design covers the dashboard, workout plans and list, workout detail and tracking, step counter, progress analytics, diet suggestions and navigation. Colours, spacing and components in the app follow the design tokens defined there.

5 Application screens

All screenshots in this section were captured from the release build running on a Samsung Galaxy M35 5G with real sensor data.

5.1 Fitness dashboard (Home)

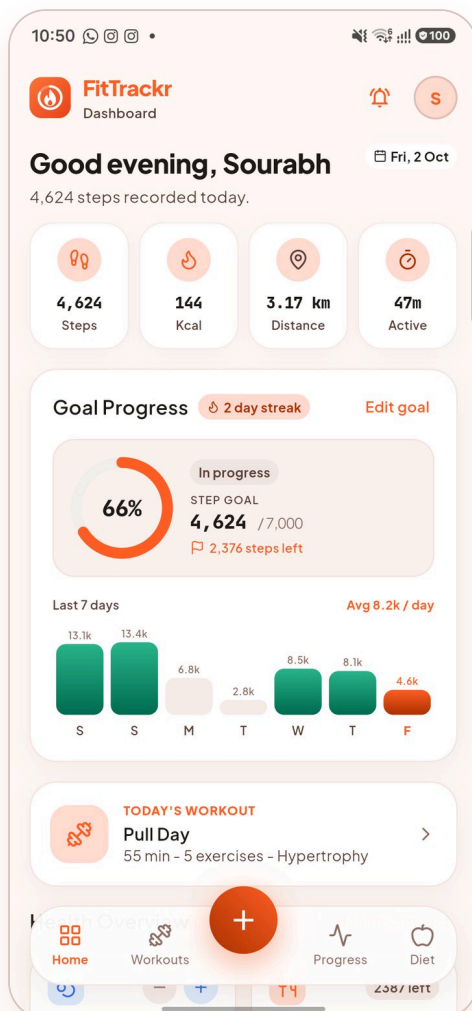


Figure 3. Dashboard: metrics, goal ring, 7-day chart

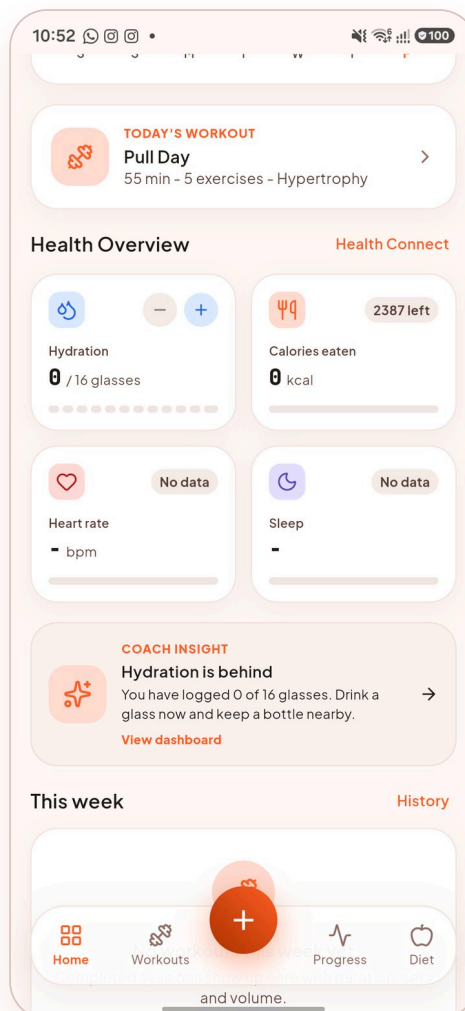


Figure 4. Health overview and coach insight

What the user sees

- A greeting with today's step count and the date.
- Four metric cards: steps, kcal, distance and active minutes.
- Goal progress: a ring with % complete, steps left and the current streak, plus a 7-day bar chart with the daily average.
- Today's workout from the weekly plan, water and calories eaten, Health Connect vitals and a coach tip.

How it's built

- **InsightsScreen** is a **ConsumerWidget** that watches **todayStepProvider**, **stepHistoryProvider(7)**, **todayWorkoutProvider** and **waterTodayProvider**.
- **.when(data, loading, error)** handles async states; **RefreshIndicator** re-reads the sensor.
- Unmeasured vitals show "No data" instead of invented values.
- **CoachInsight** picks the most useful tip (e.g. hydration behind, workout scheduled).

5.2 Workout plans and workout detail

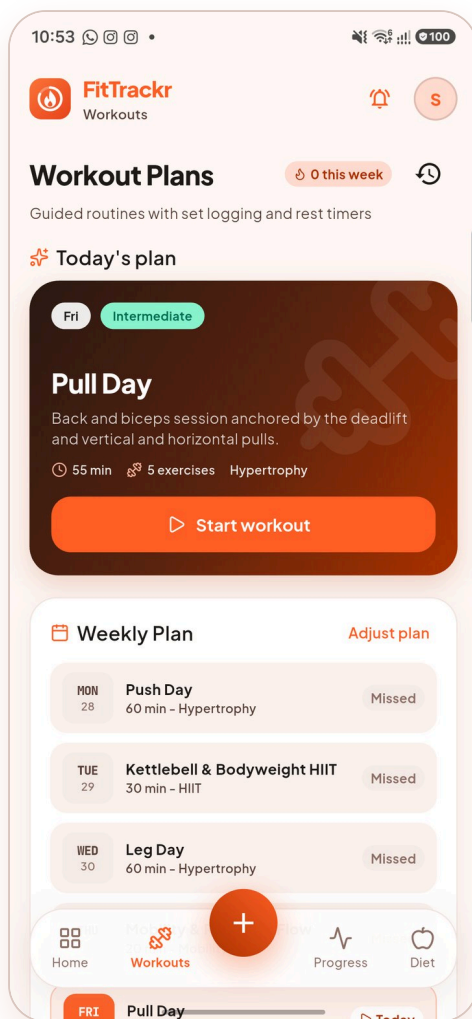


Figure 5. Today's plan and the weekly schedule

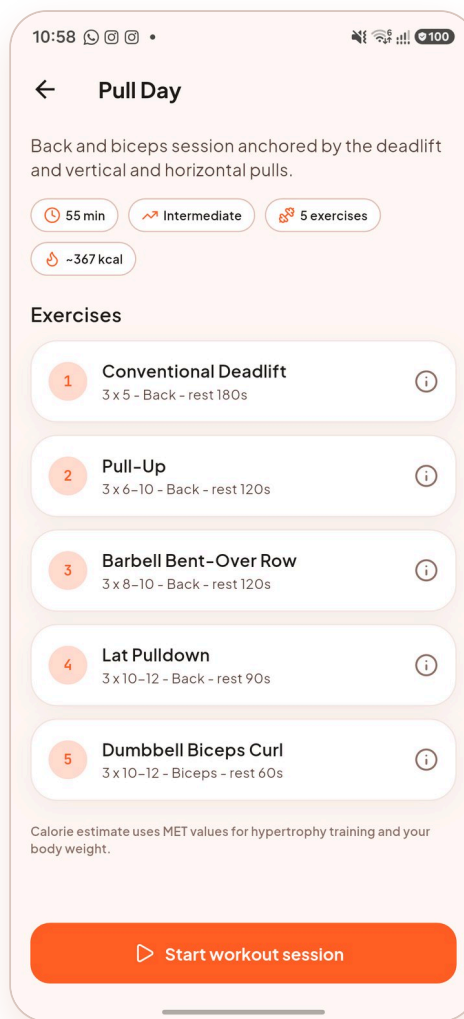


Figure 6. Workout detail with ordered exercises

Workout plans

- Hero card for today's plan with category, duration and a Start button.
- Weekly plan showing each day's workout and its status (Missed, Today, Done). Users can reassign days.
- Workout library with **ChoiceChip** filters: All, Strength, Hypertrophy, HIIT, Cardio, Mobility, Beginner. Beginner filters by difficulty; the rest by category.
- History icon opens past sessions.

Workout detail

- Receives the selected **Workout** through its constructor.
- Badges for duration, difficulty, exercise count and estimated calories ($\text{MET} \times \text{body weight} \times \text{time}$).
- Each exercise shows sets $\times$ reps, target muscle and rest time; the info icon opens setup, execution and common-mistake cues.
- Start workout session creates a session row in SQLite and opens the tracker.

5.3 Workout tracking (active session)

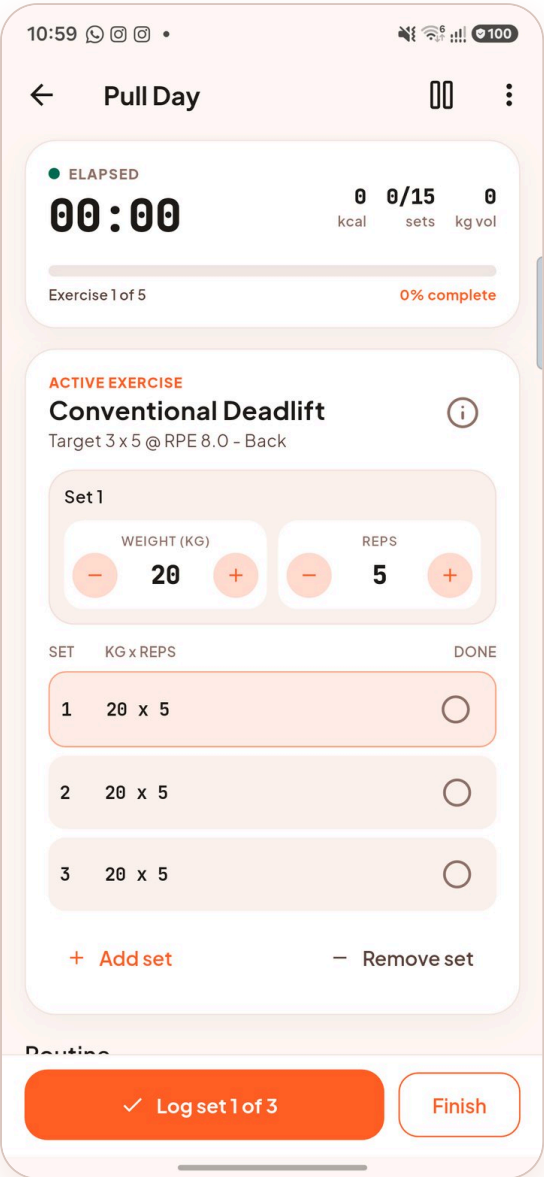


Figure 7. Live set logging

The active session screen is where progress is recorded. It shows the elapsed time, calories, sets completed and volume at the top, the current exercise with weight and reps steppers, and the list of sets with a tick for each.

Behaviour

- **Log set** marks the set complete, saves it to SQLite, and starts a rest countdown (with +30 s and Skip).
- When every set of an exercise is done, the session moves to the next unfinished exercise automatically.
- Weight changes carry over to later sets of the same exercise. Sets can be added or removed.
- Pause stops the clock. Leaving the screen keeps the session running, and it can be resumed from a banner on any tab. Discard and Finish ask for confirmation.
- **Finish** saves duration, volume, sets and calories, detects new personal records, and opens a summary with XP earned and a share button.
- Weights are prefilled from the last time the user did that exercise.

State handled by `ActiveWorkoutNotifier`

Field	Purpose
<code>setLogs</code>	Map of exercise → list of immutable <code>WorkoutSetLog</code>
<code>currentExerciseIndex</code>	Exercise in focus
<code>startedAt</code> , <code>pausedTotal</code>	Elapsed time = now – start – paused
<code>restEndsAt</code>	Rest timer end time

Time is computed from timestamps rather than a running counter, so the clock stays correct even if the app goes to the background.

5.4 Step counter and progress analytics

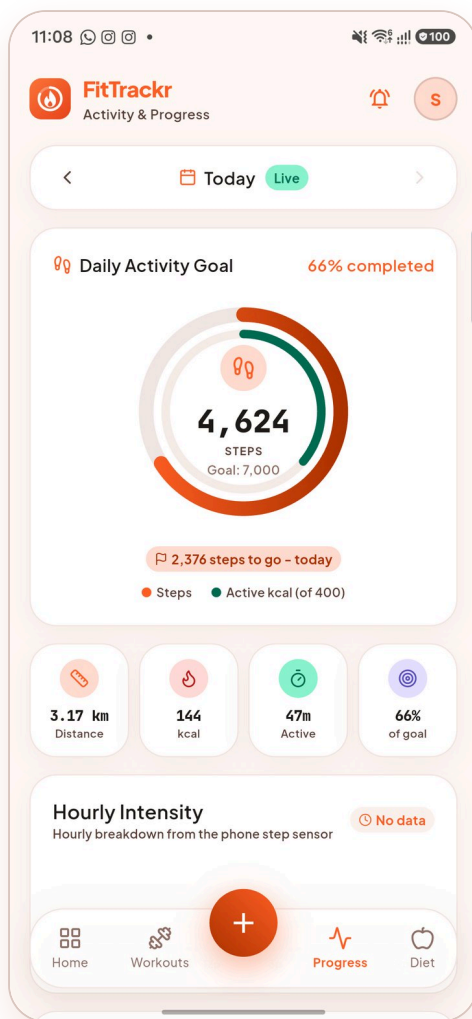


Figure 8. Step counter with daily goal ring

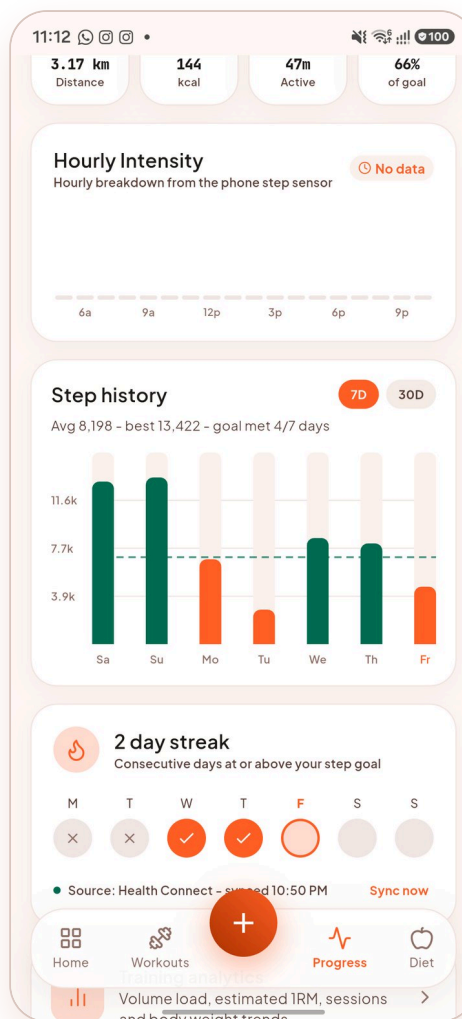


Figure 9. Step history chart and streak

Step counter (Progress tab)

- An animated dual ring (**CustomPainter** + **TweenAnimationBuilder**) for steps and active kcal against the goal.
- Distance, kcal, active minutes and % of goal; an hourly intensity chart.
- Browse previous days with the date switcher; "Live" marks today.
- Step history as a 7 or 30-day **fl_chart** bar chart with a dashed goal line, average, best day and days the goal was met.
- Streak card with the week's goal days and the data source (phone sensor or Health Connect).

Training analytics

Opened from a card on the Progress tab. It shows lifetime totals and three charts built from SQLite data:

- **Weekly volume load:** sessions grouped into the last 8 weeks (bar chart).
- **Estimated 1RM trend** per exercise using the Epley formula, with chips to pick the exercise (line chart).
- **Body weight** history, with a button to log today's weight.

Chart data preparation happens in Dart: rows from the repository are grouped by day or week, then mapped to **BarChartGroupData** and **FLSpot** objects.

5.5 Diet suggestions and meal tracking

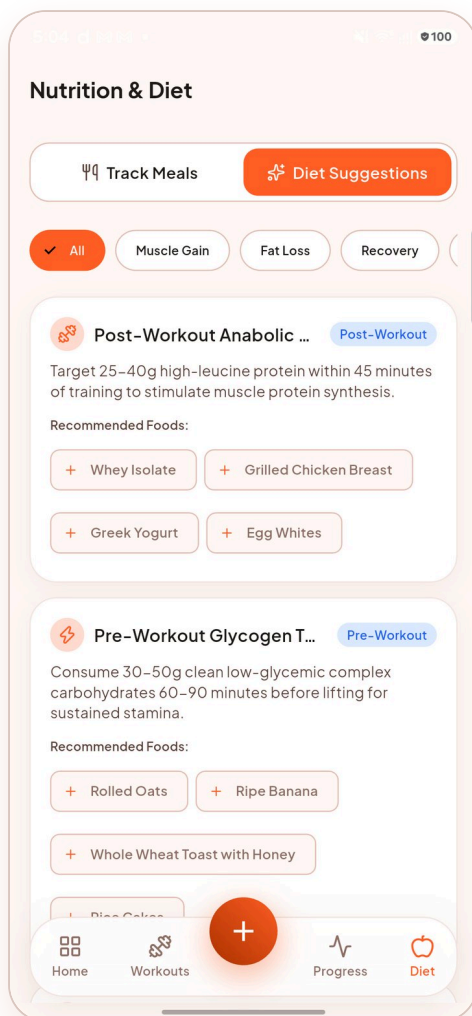


Figure 10. Diet suggestions with category filters

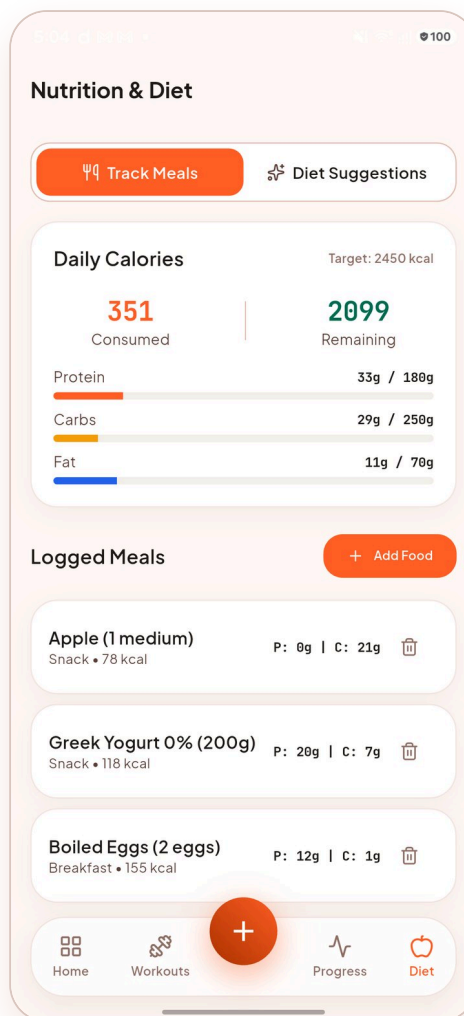


Figure 11. Daily calories, macros and logged meals

Diet suggestions

- Two tabs at the top: Track Meals and Diet Suggestions.
- Suggestion cards for Muscle Gain, Fat Loss, Recovery and Energy, each with a timing badge (Pre-Workout, Post-Workout, Morning, Night), a recommendation and foods.
- Filters are **ChoiceChips**; foods are **ActionChips** in a **Wrap**. Tapping a food opens the food search for it.

Meal tracking

- Daily calorie target and macro bars (protein, carbs, fat) from the user's BMR, activity level and goal.
- Meals grouped by Breakfast, Lunch, Dinner and Snack, with delete; a date switcher to view past days.
- Food search sheet: offline food list, recent foods and online Open Food Facts search; a servings stepper and meal slot before adding.
- Each change invalidates the dependent providers so totals update immediately.

5.6 Profile, achievements and settings

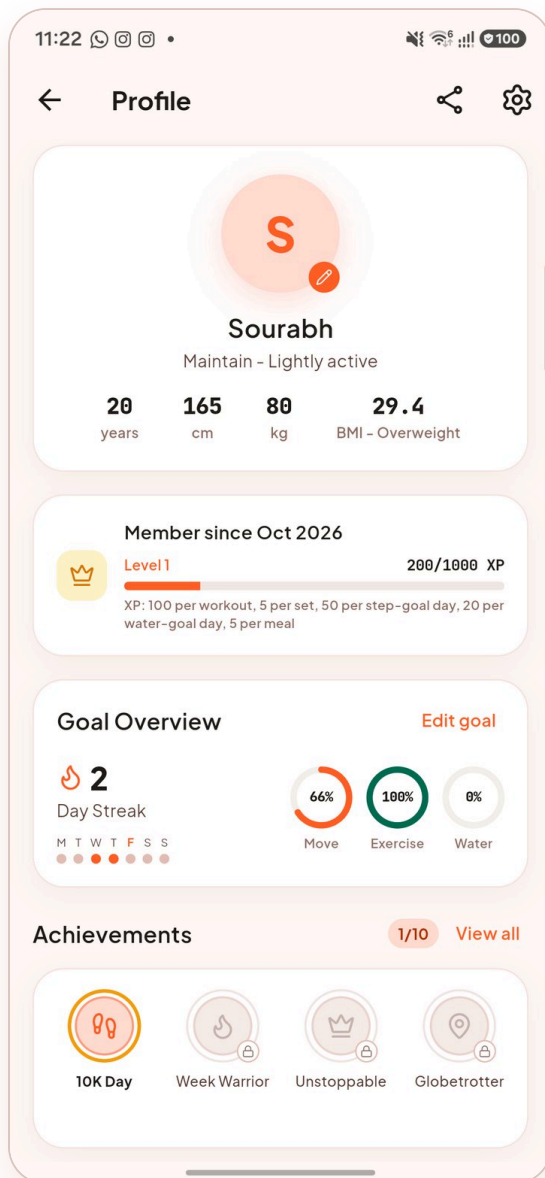


Figure 12. Profile with level, goals and achievements

The profile summarises the athlete and rewards consistency. Everything shown is computed from the local database by `userStatsProvider` and `achievementsProvider`.

Contents

- Age, height, weight and BMI with its category; goal and activity level.
- Level and XP bar. $XP = 100 \text{ per workout} + 5 \text{ per set} + 50 \text{ per step-goal day} + 20 \text{ per water-goal day} + 5 \text{ per meal}$; $\text{level} = 1 + XP \div 1000$.
- Goal overview: day streak, weekday dots, and Move / Exercise / Water rings.
- 10 achievements with progress, e.g. First Workout, 10K Day, Week Warrior, Hydration Hero, 10 Tonne Club, Globetrotter.
- Lifetime stats and personal records (best estimated 1RM per exercise).

Settings

- **Edit profile** reuses the onboarding form: `Form`, `GlobalKey<FormState>`, `TextFormField` validators (e.g. age 13–100, height 100–250 cm, step goal 1,000–50,000) and digit-only input formatters.
- Step and water goals, weekly workout plan, reminders (daily workout time, hydration, evening step check-in).
- Data sources: phone sensor status and Health Connect connection.
- Privacy note and **Reset all data**, which clears every table in one transaction.

6 Data models and database

Records are stored in a local SQLite database (`fittrackr_v1.db`, schema version 2) with foreign keys, indexes and an upgrade migration. Small settings live in `SharedPreferences`.

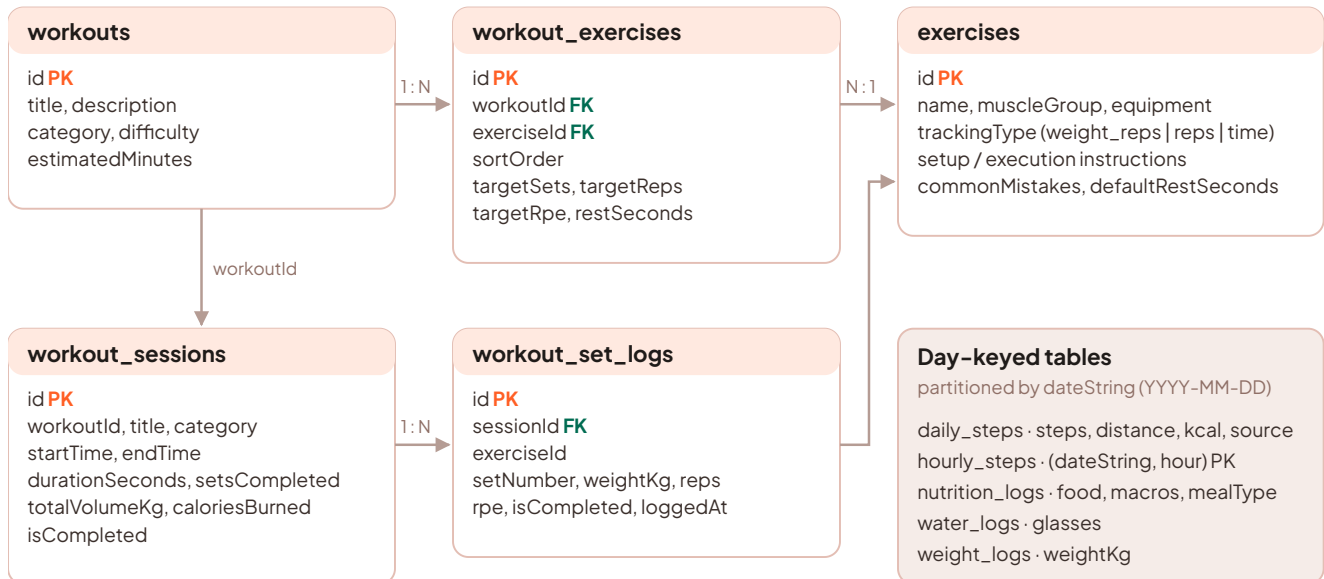


Figure 13. Database schema. Arrows point from the referenced table to the referencing one.

Dart models

Model	Purpose and key members
<code>Workout</code> , <code>Exercise</code> , <code>WorkoutExercise</code>	Catalogue. <code>fromMap</code> factories, <code>totalSets</code> , <code>targetLabel</code> ("3 x 8–10"), <code>TrackingType</code> enum.
<code>WorkoutSetLog</code>	One set; immutable with <code>copyWith</code> ; <code>volume</code> = weight × reps.
<code>WorkoutSession</code> , <code>WorkoutSummary</code> , <code>PersonalRecord</code>	Completed sessions, post-workout result (new records, XP), best e1RM per exercise.
<code>DailyStepRecord</code> , <code>HourlyStepBucket</code>	Daily totals with source (sensor or Health Connect) and hourly distribution.
<code>NutritionItem</code> , <code>MacroTargets</code>	Logged foods with <code>scaled(servings)</code> ; daily calorie and macro targets.
<code>DietSuggestion</code>	Category, timing, recommendation and foods.
<code>UserSettings</code>	Profile and preferences; computes BMR, TDEE, calorie target, BMI and macros. <code>FitnessGoal</code> enum with an extension for labels.
<code>UserStats</code> , <code>Achievement</code> , <code>HealthSnapshot</code>	Lifetime totals, XP and level; badge progress; optional vitals (nullable = not measured).

7 State management and core logic

Riverpod keeps state outside the widget tree. Widgets watch providers and rebuild only when the data they use changes.

Provider type	Examples	Role
Provider	workoutRepositoryProvider, stepRepositoryProvider	Supplies repositories (dependency injection)
Notifier	settingsProvider, activeWorkoutProvider, shellTabProvider	Synchronous state with methods that change it
AsyncNotifier	todayStepProvider, waterTodayProvider, scheduleProvider	State loaded asynchronously, exposed as loading / data / error
FutureProvider (+ .family)	allWorkoutsProvider, userStatsProvider, stepHistoryProvider(7), mealsForDateProvider(day)	Read-only data from the database, cached per parameter

After a write, dependent providers are refreshed with `ref.invalidate(...)`. Settings loaded before start-up are injected with `overrideWithValue`.

Workout completion logic (abridged from `workout_providers.dart`)

```
void completeSet(String weId, int setIndex) {
  final s = state; final we = _we(weId);
  if (s == null || we == null) return;
  final logs = Map<String, List<WorkoutSetLog>>.from(s.setLogs);
  final sets = List<WorkoutSetLog>.from(logs[weId] ?? const []);
  final updated = sets[setIndex].copyWith(isCompleted: true);
  sets[setIndex] = updated; logs[weId] = sets;
  ref.read(workoutRepositoryProvider).upsertSet(s.sessionId, updated); // save to SQLite
  // ...move to the next unfinished exercise when all sets are done
  state = s.copyWith(setLogs: logs, currentExerciseIndex: nextIndex,
    restEndsAt: DateTime.now().add(Duration(seconds: we.restSeconds)));
}
```

Formulas (`lib/core/utils/fitness_calc.dart`, `user_settings.dart`)

Calculation	Formula
Steps today	offset + (sensor reading – start-of-day baseline); a drop in the reading means a reboot, so earlier steps move into the offset
Distance, walking kcal	stride = height × 0.415 (0.5–1.0 m); distance = steps × stride; kcal = km × weight × 0.57
Goal progress, streak	min(steps ÷ goal × 100, 100); streak = consecutive goal days up to yesterday, +1 if today is met
BMR (Mifflin–St Jeor)	10 × weight + 6.25 × height – 5 × age + 5 (male) or – 161 (female)
Calorie target	BMR × activity factor (1.2–1.725), –500 to lose or +300 to gain; minimum 1,500 / 1,200 kcal
Macros	protein 1.6–2.0 g/kg; fat 25% of kcal ÷ 9; carbs = remaining kcal ÷ 4
Workout kcal	MET × weight × hours (HIIT 8, cardio 7, strength 5, mobility 2.5)
Volume, 1RM	Σ weight × reps of completed sets; Epley 1RM = weight × (1 + reps ÷ 30)

8 Integration, styling and theming

Integrations

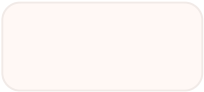
Integration	Implementation
Device step data	PedometerService requests <code>ACTIVITY_RECOGNITION</code> , listens to the hardware step counter stream, keeps a daily baseline in <code>SharedPreferences</code> and broadcasts updates. Totals are saved at most every 15 seconds, with hourly buckets.
Health Connect	Optional, via the health package: reads steps (with a 7-day backfill), heart rate, sleep and SpO2. The app shows the higher of sensor and Health Connect steps.
Local storage	SQLite for all records; <code>SharedPreferences</code> for profile, goals, reminders and the weekly plan.
Diet data service	Open Food Facts search API (below) plus an offline catalogue of 34 common foods and recently logged foods.
Notifications	Daily reminders scheduled with zonedSchedule ; an instant notification when the step goal is reached.

```
// nutrition_api_service.dart (abridged)
final uri = Uri.parse(_searchBaseUrl).replace(queryParameters: {
  'search_terms': q, 'json': '1', 'page_size': '25',
  'fields': 'code,product_name,brands,nutriments',
});
final response = await _client.get(uri, headers: _headers).timeout(const Duration(seconds: 12));
if (response.statusCode != 200) throw Exception('Food database returned ${response.statusCode}');
final data = json.decode(utf8.decode(response.bodyBytes)); // then map to NutritionItem
```

The UI catches errors (including timeouts) and shows a message, so the app never crashes without internet.

Material 3 theme

AppTheme.lightTheme sets **useMaterial3**: **true**, a custom **ColorScheme**, and themes for app bars, cards, buttons, inputs, chips, list tiles, dialogs, bottom sheets, snack bars, sliders, switches, progress indicators and page transitions.

					
Primary #FF5F25	Primary dark #AD3300	Success #006C52	Sleep #594EBD	Canvas #FFF8F5	Headline #1F1B19

Plus Jakarta Sans

Headings and body text, weights 400–800, bundled in **assets/fonts**.

4,624 · 00:00

JetBrains Mono for numbers, timers and metrics.

States: coral for active and in-progress, green for completed and goal met, muted for missed or no data. **Responsive**: text scaling clamped to 0.85–1.25, **Expanded**, **FittedBox**, **Wrap** and **SafeArea**; portrait orientation.

9 Testing, CI/CD and deliverables

Automated tests (19)

File	What is tested
<code>test/logic_test.dart</code>	Mifflin-St Jeor BMR, Epley 1RM, target parsing ("6-8", "30s"), distance scaling with height, date keys across months, macro targets adding up to the calorie target, calorie floor, longest streak, number formatting
<code>test/database_test.dart</code>	Runs on an in-memory SQLite database: every seeded workout has exercises; session volume, personal records and history; steps, hourly buckets, meals and water persist
<code>test/widget_test.dart</code>	First launch shows onboarding; onboarding validates the name field; weekly bars render one label per day; goal card shows remaining steps; diet suggestions filter by category; coach insight prioritises a scheduled workout

CI/CD pipeline

The GitHub Actions workflow `.github/workflows/android-release.yml` runs on pushes to `main`, version tags and pull requests:



Deliverables checklist

Deliverable	Evidence in this project	Status
Figma design	Dashboard, workout plans/list, detail/tracking, step counter, progress analytics, diet suggestions, navigation (section 4)	Done
UI / Widgets	Workout cards, exercise lists, progress indicators, step counter ring, charts, suggestion cards, filter chips, tabs, floating navigation (section 5)	Done
Styling / Theming	Material 3 theme, status states, typography, spacing, Lucide icons, chart styling, responsive behaviour (section 8)	Done
Dart / Flutter logic	Workout, step, progress and diet models; completion tracking; step aggregation; progress calculations; chart data; Riverpod state; navigation (sections 6-7)	Done
Integration	Hardware step sensor and Health Connect; local SQLite and SharedPreferences storage; Open Food Facts API for food data	Done
Responsive prototype	Full journey from selecting a workout and recording sets, through steps and charts, to diet suggestions; release APK on GitHub	Done

10 Limitations, future scope and conclusion

Current limitations

- **Single device, no account.** Data stays on the phone; there is no cloud backup or sync.
- **Android first.** Step counting and Health Connect are Android features; iOS would need HealthKit.
- **Light theme only,** with a custom navigation dock instead of the Material 3 **NavigationBar**.
- **Curated diet suggestions.** Suggestions are a fixed set filtered by goal; calorie and macro targets are personalised.
- **Hourly steps are best-effort.** The sensor reports while the app listens, so steps taken while it is closed are attributed to the hour the app reopens.
- **Navigation 1.0.** Screens use **Navigator** rather than **go_router**, so there are no deep links.

Future scope

- Firebase Authentication and Firestore for accounts and cloud sync.
- Dark mode using **ColorScheme.fromSeed** with a dark variant.
- **go_router** for named routes and deep links from notifications.
- Personalised meal plans from the user's goal and logged history.
- Custom workout builder and progressive-overload suggestions.
- Integration tests with **integration_test**, and an iOS build with HealthKit.

Conclusion

FitTrackr meets the goals of Problem Statement 59. It brings workout planning and tracking, step counting, progress analytics and diet guidance into one Flutter app with a consistent Material 3 interface that follows the Figma design. The project applies the main topics of the course: Dart null safety, OOP and async programming; stateless and stateful widgets; responsive layouts; Riverpod state management; navigation and validated forms; local persistence with SQLite and SharedPreferences; REST API integration; testing; and an automated build pipeline. It runs on a real Android phone with real sensor data and works fully offline.

References

- Flutter documentation: docs.flutter.dev · Dart language: dart.dev/language
- Riverpod: riverpod.dev · Material 3 for Flutter: m3.material.io/develop/flutter
- sqflite, pedometer, health, fl_chart, http packages: pub.dev
- Open Food Facts API: world.openfoodfacts.org · Android Health Connect: developer.android.com/health-connect
- Mifflin M.D. et al. (1990), a new predictive equation for resting energy expenditure. Epley B. (1985), one-rep max estimate. Compendium of Physical Activities (MET values).
- Napoli, M. L. *Beginning Flutter: A Hands On Guide to App Development*. Cybellium, *Mastering Flutter*.



FitTrackr v1.1.0

Source: github.com/YadavSourabhGH/FitTracker · APK: [GitHub Releases](#)